

Sketch - Notebook for Programming

Made by AndreiDani
<3 - de la Iari ;)



Cuprins:

1. Introducerea in problemă (idea unui controler)
2. Controlerul Proporțional - Integral - Derivat (aka. PID)
 - a. Definiții introductive
 - b. Calcularea corecției
 - c. Filtrarea zgomotului (aka. noise filtering)
 - d. Zona moartă a erorii (optimizare)
 - e. Limitarea integralei (aka. integral windup)
 - f. Grafice - noise filtering
3. Object Stall Detection - detectarea coliziunilor pentru motoare
4. Deplasări față / spate
 - a. Procesul de transformare al distanței
 - b. De ce folosim un controler PID?
5. Viraje la stânga și la dreapta - de spin și de pivot (pe o roată)
 - a. Transformarea vitezei din grade / secundă
 - b. Optimizări pentru a îmbunătăți precizia (unghiul virtual)
6. Schimbarea între runuri și cum am optimizat acest aspect :))
7. Links, Github page, QR codes ;)

1. Introducerea în problemă

Pentru a mișca robotul folosim 4 funcții principale:

- Deplasare față / spate
- Viraj stânga / dreapta

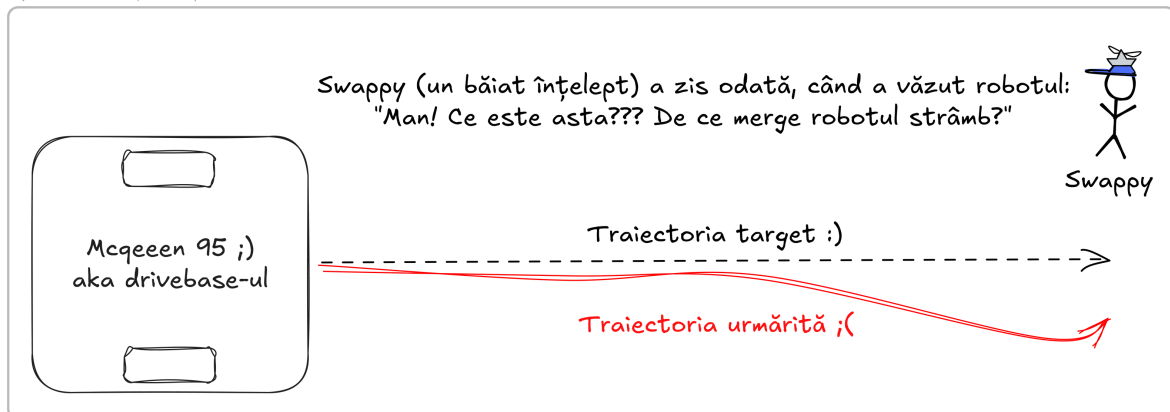
Acum vine problema. Cum pot fi acestea implementate cu precizia mare?

---> *The true journey* <---

Nu putem aplica o viteză constantă pe setul de roți, cu speranța să ajungem în același loc de 10 ori la rând. (dacă se întâmplă asta este un miracol)

Trebuie să dezvoltăm o metodă ca robotul **să își mențină traiectoria inițială** sau **să revină** la aceasta, după **devieri laterale**.

Lipsa de acuratețe în mișcările robotului



O metodă pentru a ajuta robotul să își mențină traiectoria este folosind un algoritm stil **Feed Forward**, dar este **prea complex** pentru scopul nostru. (pentru doritori, merita documentat acest algoritm :D)

O metodă **mai simplă** pentru a ajuta robotul să se întoarcă la traiectoria sa este a modifica vitezele motoarelor simetric (similar cu un viraj). Valoarea cu care trebuie modificata poate fi calculată cu ajutorul **unui controler**.

2. Controlerul PID

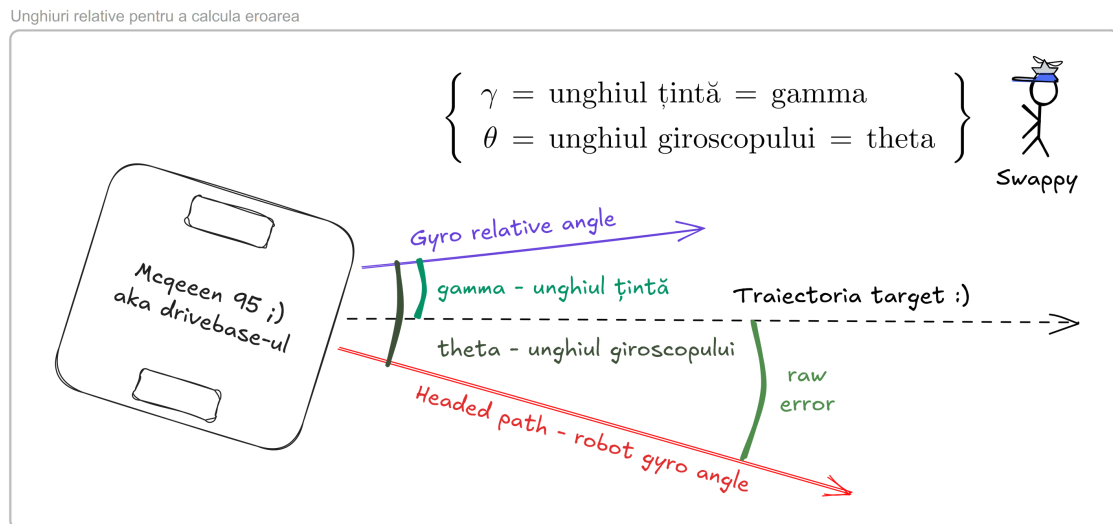
Controlerele Proporționale - Integrale - Derivate (PID) se folosesc cand avem o valoare *INPUT* și una *TARGET*, la care *trebuie inputul să ajungă*.

a. Definitii introductive

Înainte de a intra în detalii trebuie **definite** anumite **constante, variabile și funcții** care ajută la înțelegerea **scopului controlerului** și la **explicarea acestuia**. Pentru început trebuie să înțelegem care este *inputul* primit.

$$\left\{ \begin{array}{l} \gamma = \text{unghiul țintă} \\ \theta = \text{unghiul giroscopului} \end{array} \right\} \rightarrow \left\{ \begin{array}{l} E_{raw}(t) = \gamma - \theta \\ E_{raw}(t) = \text{eroarea de la traiectorie} \\ E(t) = E_{raw}(t) = \text{eroarea finală} \end{array} \right\}$$

Pentru a înțelege mai ușor ce reprezintă aceste variabile am făcut **urmatoarea diagrama** explicativă. :DD



În cazul nostru, **inputul** este egal cu **theta**, iar valoarea **target** este egală cu **gamma**. (de reținut că acestea se pot modifica pentru diferite aplicații ale controlerului în anumite industrii / competiții)

De asemenea, folosim următoarele **constante** și **limite** pentru a calcula **corecția** și pentru a face **output-ul și traseul robotului mai precis**. :)

$$\begin{aligned}\omega_{variable} &= \text{limită pentru o variabilă} \longrightarrow \omega_{variable} \in [0, 2^{20}] \\ \omega_{variable} &\rightarrow \text{variable} \in [-\omega_{variable}, +\omega_{variable}] \\ K_p, K_i, K_d &= \text{constantele folosite în controlerul PID} \\ \alpha &= \text{constantă folosită pentru filtrarea zgomotului giroscopului} \\ \epsilon_{error} &= \text{intervalul erorilor acceptabile} \longrightarrow \epsilon_{error} \in [0, 10] \text{ grade} \\ \epsilon_{error} &\rightarrow \text{acceptabilă} \Leftrightarrow (E(t) \in [-\epsilon_{error}, +\epsilon_{error}]) \\ \omega &= \text{omega} \quad | \quad \alpha = \text{alpha} \quad | \quad \epsilon = \text{epsilon}\end{aligned}$$

b. Calcularea corecției

Precum am zis și la început, **un controler PID** ajută la **aducerea unei valori (input) la o valoare dorită (target)**. Eroarea este definită în următorul fel:

$$\left\{ \begin{array}{l} E_{raw}(t) = \gamma - \theta \\ E_{raw}(t) = \text{eroarea de la traiectorie} \\ E(t) = E_{raw}(t) = \text{eroarea finală} \end{array} \right\}$$

Acum vom explica **rolul fiecarui termen** din **controlerul P.I.D.**

Proportională este responsabilă pentru corectarea erorii curente. Aceasta accelerează timpul de raspuns și stabilizează traiectoria robotului, oferind o corecție proporțională cu magnitudinea erorii. (de aici îi vine și numele)

$$P(t) = K_p \cdot E(t) \longrightarrow \text{proporțional cu eroarea}$$

De remarcat: Acest termen nu este în suficient pentru a putea stabili traectoria și pentru a face robotul să se întoarcă la traectoria inițială, dar este un punct de plecare foarte bun pentru toate echipele. :DD

Integrala este responsabilă pentru corectarea erorilor acumulate. Aceasta ajută la eliminarea erorilor Steady-State (care nu pot fi corectate de proporțională și de derivată, odata ce se stabilizează sistemul), astfel aducând inputul la valoarea target.

$$I(t) = K_i \cdot \int_0^t E(x) dx \longrightarrow \text{acumularea totală a erorii}$$

Derivata este folosită pentru a prezice următoarea eroare. Aceasta ajută la minimizarea timpului de oscilare și a overshoot-ului, astfel oferind stabilitate sistemului și minimizând offset-ul de la traiectoria target.

$$D(t) = K_d \cdot \frac{d}{dt} E(t) \longrightarrow \text{rata de schimbare a erorii}$$

Corecția este suma tuturor termenilor, iar aceasta este adăugată sau scăzută din viteza motoarelor. (plus pe un motor, minus pe celalalt)

$$C(t) = K_p \cdot E(t) + K_i \cdot \int_0^t E(x) dx + K_d \cdot \frac{d}{dt} E(t) \longrightarrow \text{corectarea finală}$$

De remarcat: În unele cazuri este optimă folosirea unor controlere particulare, precum un controler PI (proporțional - integral) sau un controler PD (proporțional - derivat).



"I'm going to program this overnight."
--> famous last words of Swappy <--

În concluzie: Controlerele PID ne ajută să corectăm traseul robotului în timp real și ne asigură mișcări precise, rapide și robuste.

c. Filtrarea zgomotului - noise filtering

Filtrarea zgomotului (aka. noise filtering) este un algoritm care ajută la omiterea schimbărilor nesemnificative în eroare care nu afectează traiectoria robotului, acestea venind de cele mai multe ori din factori externi.

Dacă acestea nu sunt eliminate, controlerul PID le ia în considerare, astfel instabilizând traiectoria robotului semnificativ pe distanțe mari.

Sunt mai multe metode de a implementa noise filtering, fiecare având avantaje, dezavantaje și rezultate finale diferite. Cea mai ușoară de înțeles este de a folosi o funcție de gradul întâi (aka. exponential smoothing).

$$\alpha \in [0, 1] \longrightarrow \text{constantă pentru filtrarea erorii}$$

Algoritmul de noise filtering atribuie două costuri complementare: α pentru eroarea curentă, iar $(1 - \alpha)$ pentru eroarea filtrată veche.

În următoarea schemă este prezentat algoritmul ca o funcție.

$$E_f(t) = \alpha \cdot E_{raw}(t) + (1 - \alpha) \cdot E_f(t - 1) \longrightarrow \text{eroarea filtrată}$$

De remarcat: Acum eroarea finală este egală cu eroarea filtrată.

$$E(t) = E_{raw}(t) \text{ sau } E_f(t) \quad \Leftrightarrow \quad \alpha = 1 \rightarrow E_f(t) = E_{raw}(t)$$

Note: Pentru a înțelege mai ușor scopul funcțiilor, am făcut mai multe grafice în care am pus eroarea raw și output-ul algoritmului de noise filtering cu multiple constante setate, pentru a demonstra felul funcționării acestuia. (aceste grafice apar la finalul capitolului despre controlere PID)

d. Zona moartă a erorii - error deadzone

Aceasta este tot o optimizare și îmbunătățire a controlerului PID, care ține de modificarea erorii curente, în cazul în care este foarte mică, înainte de aplicarea algoritmului de filtrare.

$$(E_{raw}(t) \in [-\epsilon_{error}, +\epsilon_{error}]) \rightarrow E_{raw}(t) = 0$$

Note: Epsilon este definit în subcapitolul a.

e. Limitarea integralei - integral windup

Limitarea integralei constă în restricționarea integralei să se afle într-un interval anumit de valori. Această modificare asigură faptul că robotul își poate corecta traiectoria, chiar și după coliziuni.

$$I(t) = K_i \cdot \max(-\omega_i, \min(+\omega_i, \int_0^t E(x) dx)) \rightarrow \text{limitarea integralei}$$

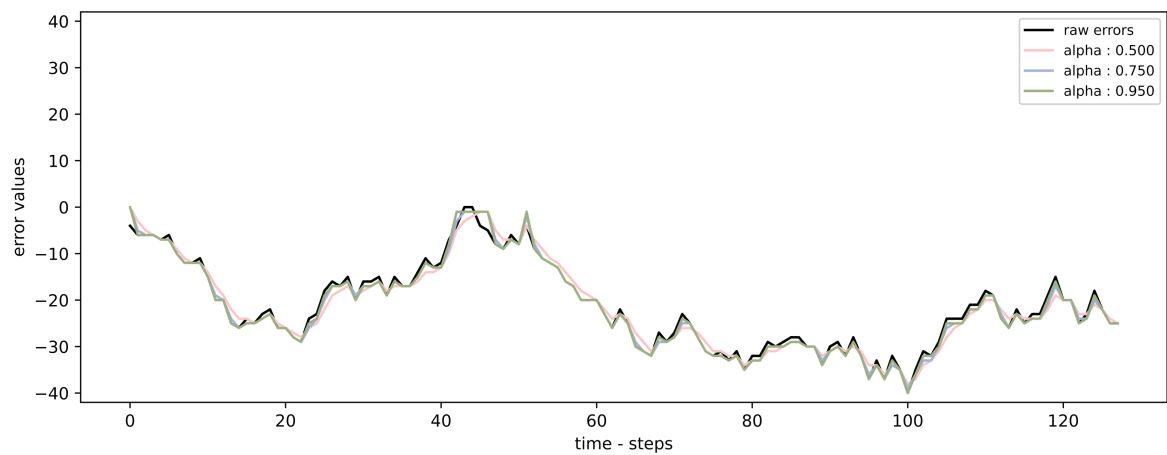
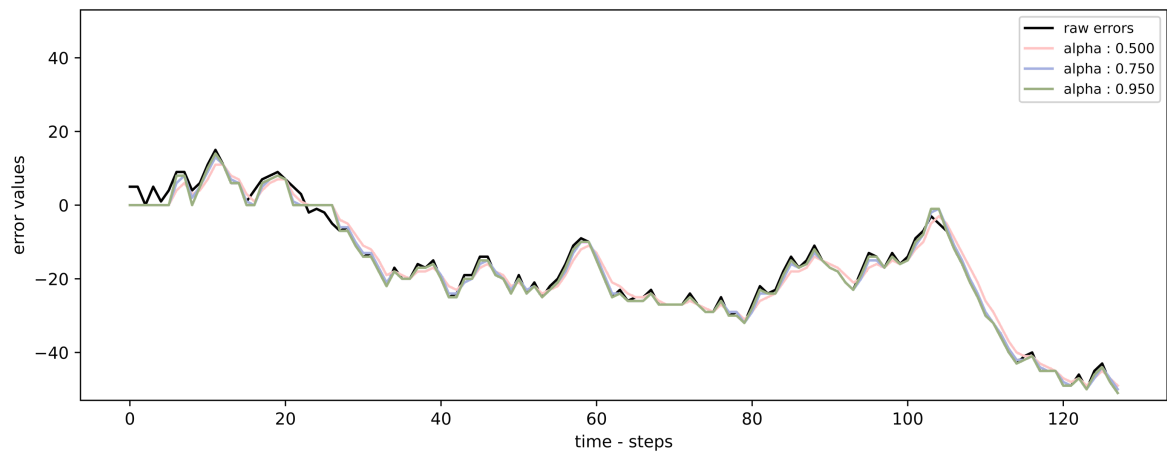
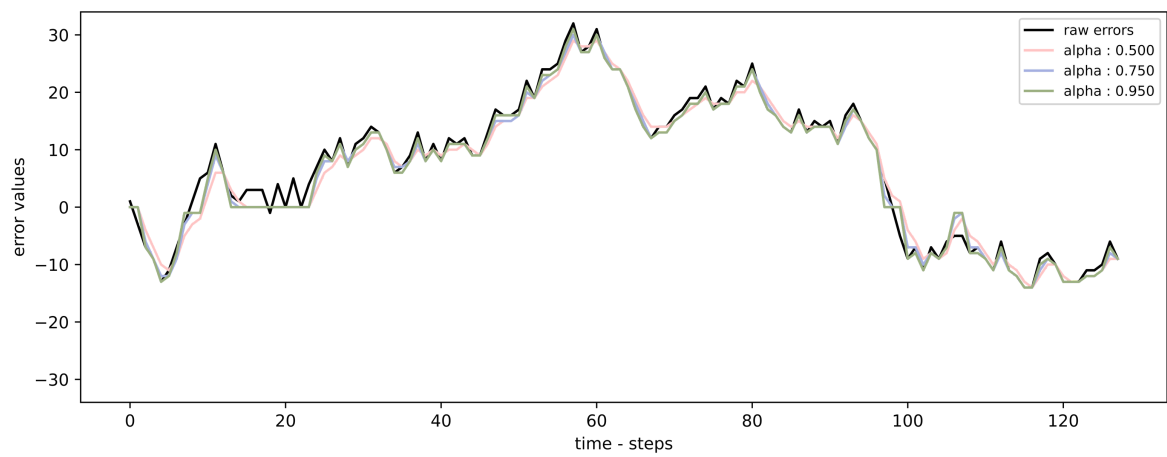
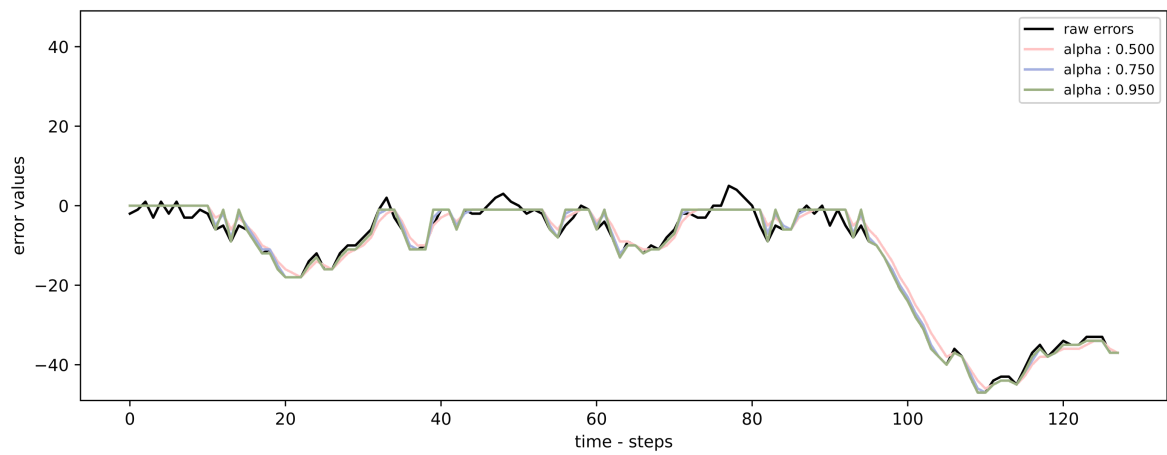
$$\omega_i \in [0, 2^{20}] \rightarrow I(t) \in (K_i \cdot [-\omega_i, +\omega_i])$$

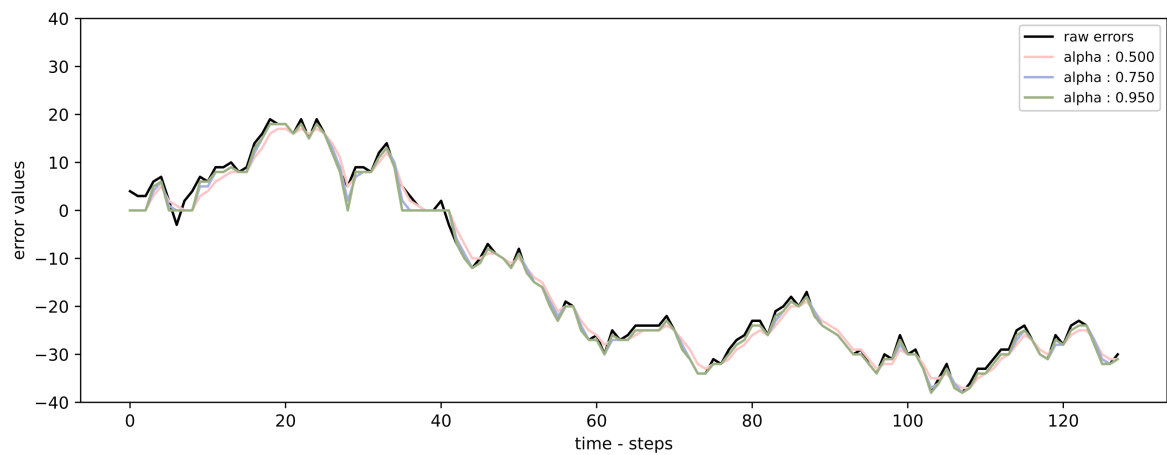
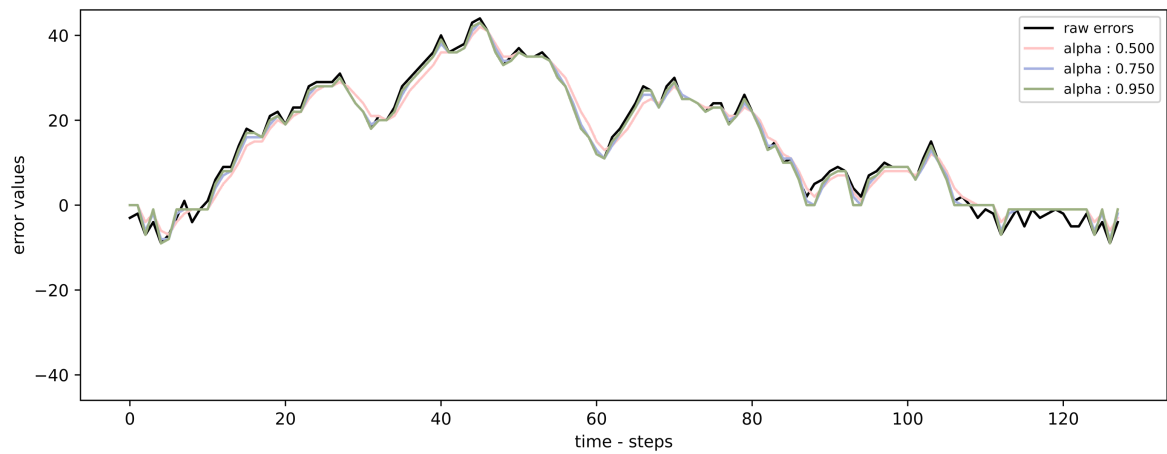
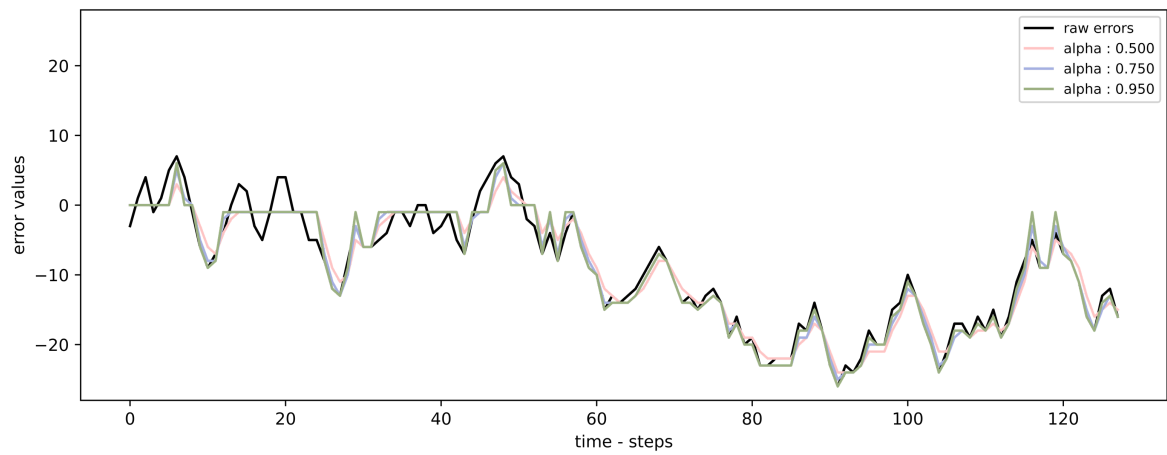
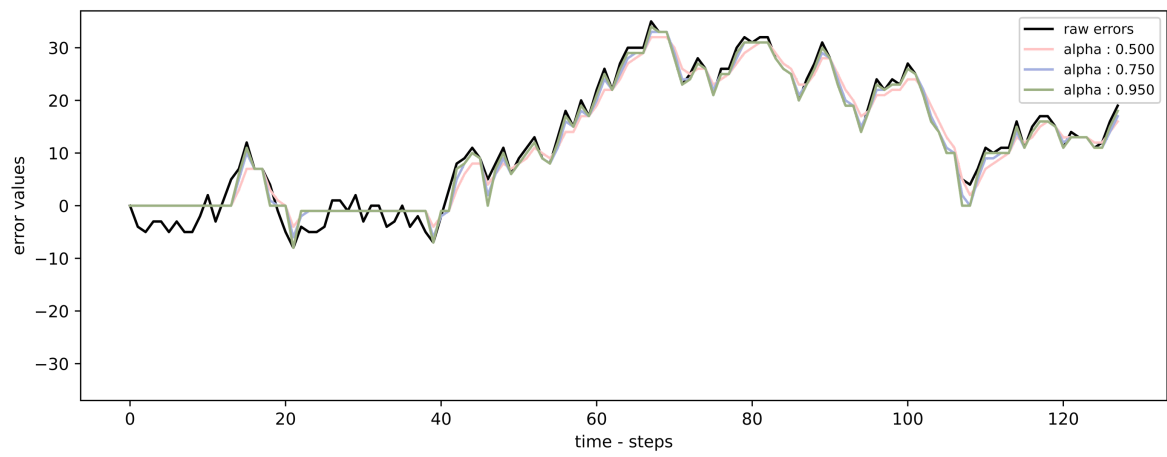
Note: Un scop secundar al acestei modificări este evitarea overflow-ului din cauza arhitecturii procesorului. (de exemplu depășirea valorii maxime rezultă în schimbarea semnului din + în -, lucru care destabilizează sistemul)

f. Grafice - noise filtering & error deadzone

Precum am promis, urmează să prezentăm o serie de grafice diferite (făcute de prietenul nostru Swappy) în care demonstrăm corectitudinea algoritmului de noise filtering și optimizarea legată de error deadzone.

Note: Pentru curioși graficele sunt făcute în python - matplotlib.





3. Object Stall Detection

Object Stall Detection este un algoritm care oprește motoarele în cazul detectării unei coliziuni sau în cazul în care acestea nu se mai pot mișca.

$\text{motor.stalled()} = \text{true} \Leftrightarrow (\text{motor.position()} - \text{lastt}_{\text{position}} < \text{prag})$
 $\text{motor.stalled()} =$ verifică dacă motorul s-a oprit – nu se mai poate mișca
 $\text{lastt}_{\text{position}} = \text{motor.position()} \rightarrow$ schimbată după un interval de timp

Note: Acest algoritm ajută la finalizarea tuturor funcțiilor, chiar și în cazul în care motoarele nu pot finaliza acțiunea dată și ajută la conservarea motoarelor și a bateriei, neîndând motoarele forțate.

4. Deplasări față / spate

Deplasările față / spate sunt folosite atunci când trebuie să mergem drept și pe o distanță specificată. Acestea sunt implementate cu ajutorul unui controler PID pentru a menține traiectoria robotului și cu ajutorul encoderelor integrate în motoare, care verifică dacă distanța a fost parcursă.

a. Transformarea distanței în ceva util

Pentru a ne folosi de encoderele integrate, trebuie să transformăm distanța în unitatea de măsură folosită de encoder, precum grade sau radians.

Aici vom prezenta cum transformăm distanța din mm în grade. Pentru început vom nota cu 'r' raza roții. Pentru a face regula de trei simple trebuie știut că înseamnă o unitate de măsură (mm, inch, etc) în grade.

$$c_{\text{wheel}} = \frac{360^\circ}{2\pi r} \rightarrow \text{gradele necesare pentru a ne mișca o unitate}$$

$$c_{\text{wheel}} = \frac{360^\circ}{\pi \cdot 49.5 \text{ mm}} \approx 2.314^\circ \rightarrow \text{pentru roțile noastre}$$

Deoarece știm cât înseamnă o unitate în grade putem transforma distanța din mm în grade printr-o singură și simplă înmulțire. :D

$$\begin{aligned}d_{unit} &= \text{distanța în unitatea de măsură} \\d_{degrees} &= d_{unit} \cdot c_{wheel} \longrightarrow \text{distanța transformată}\end{aligned}$$

b. De ce folosim un controler PID?

Precum știți, un controler PID este util pentru a aduce o valoare input la o valoare target, iar în cazul acesta am vrea ca direcția robotului să fie aceeași cu direcția target (implicit traseul urmărit fiind același cu traiectoria target).

Note: Nu pot fi omise controlerele PID, din cauza factorilor externi care distribuie aderența pe roți neuniform, precum sistemele nesimetrice și grele, planșe alunecoase sau cu praf, roți murdare și lista poate continua.

5. Viraje la stânga și la dreapta

Virajele sunt folosite atunci când se dorește efectuarea întoarcerilor pivotante (cu o roată mobilă și una fixă) și întoarcerilor spin (roțile se mișcă în direcții opuse). Aici nu mai este nevoie de un controler PID, dar sunt necesare unele transformări și optimizări, pentru a crește precizia.

a. Transformarea vitezei

Pentru a ști durată unui viraj și cazul în care aceasta este depășită cu mult, am decis să ținem viteza în grade (pentru robot) pe secunda.

$$\begin{aligned}s_{degrees} &= \text{viteza de rotație cu o roată în } \frac{\text{grade}}{1 \text{ secundă}} \\d_{wheels} &= \text{distanța între cele două roți}\end{aligned}$$

Vrem să ajungem la regula de trei simple, la fel ca la transformarea distanței din mm / inch în grade / radians. Pentru asta este nevoie să știm cât înseamnă a se roti robotul cu un grad, dacă o roată este fixă (viraj pivot).

$$d_{path-wheels} = \frac{2\pi \cdot d_{wheels}}{360^\circ} \rightarrow \text{distanța pentru a roti robotul cu } 1^\circ$$

$$d_{speed} = d_{path-wheels} \cdot s_{degrees} \rightarrow \text{distanța pentru a ne roti cu viteza dorită}$$

Știind cu cat trebuie să se miște o roată pentru a ajunge la viteza dorită, putem transforma distanța înapoi în grade, pentru a fi aplicată la motoare.

$$s_{wheel} = d_{speed} \cdot c_{wheel} \rightarrow \text{viteza finală a motorului :D}$$

Note: Pentru a generaliza idea pe virajele spin, viteza finală se împarte la 2.

b. Unghiul virtual - optimizare

Deoarece un run constă într-o serie de mișcări față / spate, viraje de mai multe feluri și acționări de motoare este important să nu pierdem din precizie, cu cât avansăm mai mult in run.

Pentru a face asta, putem ține unghiul giroscopului și unghiul target, care va fi inițializat la începutul fiecarui run cu 0. Acum dupa fiecare viraj putem modifica unghiul target (din definițiile de la PID acesta este gamma) cu plus / minus unghiul virajului, astfel măbind precizia semnificativ.



"O să meargă robotul perfect :D"
--> Swappy <--

6. Schimbarea între runuri

În timpul meciurilor este crucial să putem ajunge la runul dorit cât mai repede, fără a mai pierde timp cu resetări inutile, care pierd din precizie și care încetinesc și drive team-ul.

Pentru a rezolva această problemă, am decis să salvăm numărul ultimului run dat în memoria internă a robotului (biții) și când inițializăm programul și clasele să pornim de la runul citit din memorie.

Note: Această optimizare a scăzut timpul selectării cu peste 200%.

7. Linkuri și pagini github

Am decis să avem totul open source și am postat programele noastre de-a lungul sezonului pe o pagină dedicată de github.



[https://github.com/AndreiDaniDev/
Unearthed-FLL-Code](https://github.com/AndreiDaniDev/Unearthed-FLL-Code)



[https://en.wikipedia.org/wiki/Propo
rtional%E2%80%93integral%E2%80%
%93derivative_controller#](https://en.wikipedia.org/wiki/Proportional%E2%80%93integral%E2%80%93derivative_controller)



x. Let swappy explain this

idk man let me think (swappy)

